# Day 99 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 15, Day 2 reference

Watch: HuggingFace Crash Course - Sentiment Analysis, Model Hub, Fine Tuning (Patrick Loeber) | BERT: Pre-training of Deep Bidirectional Transformers (Paper Explained) (Deep Learning Explainer)

`day99_fine_tuning_bert.py`, segment by segment, in the order `main()` uses things. Day 98's script is imported for the data loader, the bag-of-words model, the bootstrap and the ready-made model. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

**MATH BOX LEGEND**

**Symbols:** $\eta_t$ = learning rate at step t · $\lambda$ = L2 penalty · a, b = two models' 0/1 results on the same sentences · [CLS] = the first token's vector

## Segment 0 — Setup

### Imports and settings

**CODE**

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day98_pretrained_transformers as d98
hf_logging.set_verbosity_error(); hf_logging.disable_progress_bar()
hub_logging.set_verbosity_error(); disable_progress_bars()
BASE = "distilbert/distilbert-base-uncased"
LR, BATCH, EPOCHS, MAX_LEN = 5e-5, 16, 2, 64
WARMUP_FRAC, WEIGHT_DECAY = 0.1, 0.01
CHECKS = 8
THREADS_PER_RUN = 3
RUNS = [(250, 0), (1000, 0), (1000, 1), (1000, 2), (4000, 0)]
```

- `import day98_pretrained_transformers as d98` — Day 98's script as a module; its main() does not run on import.
- `LR, BATCH, EPOCHS, MAX_LEN = 5e-5, 16, 2, 64` — multiple assignment from a tuple: four settings in one line.
- `RUNS = [(250, 0), ...]` — a list of (training items, seed) jobs, one per parallel process.

## Segment 1 — Data

### data: one shuffle, one validation split

**CODE**

```
def data():
    Xtr, ytr = d98.sst2("train")
    Xva, yva = d98.sst2("validation")
    order = np.random.RandomState(0).permutation(len(Xtr))
    Xtr, ytr = [Xtr[i] for i in order], ytr[order]
    half = np.random.RandomState(1).permutation(len(Xva))
    dev, test = half[:len(Xva) // 2], half[len(Xva) // 2:]
    return (Xtr, ytr), ([Xva[i] for i in dev], yva[dev]), ([Xva[i] for i in test], yva[test])
```

- `np.random.RandomState(0).permutation(len(Xtr))` — the same shuffle in every process, so 'the first n items' means the same items everywhere.
- `ytr[order]` — fancy indexing reorders the labels exactly as the texts.

- `return (Xtr, ytr), (...), (...)` — a tuple of three (texts, labels) pairs, unpacked by the caller.

## Segment 2 — Fine-tuning

### lr_at: warm-up and linear decay

**MATH**

$\eta_t = \eta\ (t + 1) / w$ *for* $t < w$, $\eta_t = \eta\ (T - t) / (T - w)$ *after*

**CODE**

```
def lr_at(step, total):
    warm = max(1, int(WARMUP_FRAC * total))
    return LR * (step + 1) / warm if step < warm else LR * max(0.0, (total - step) / (total -
        warm))
```

- `max(1, int(WARMUP_FRAC * total))` — at least one warm-up step, even for very short runs.
- `x if cond else y` — a conditional expression: the warm-up branch or the decay branch.

### fine_tune: the PyTorch training loop

**CODE**

```
def fine_tune(job):
    n, seed = job
    torch.set_num_threads(THREADS_PER_RUN); torch.manual_seed(seed)
    (Xtr, ytr), (Xd, yd), (Xt, yt) = data()
    Xtr, ytr = Xtr[:n], ytr[:n]
    tok = AutoTokenizer.from_pretrained(BASE)
    model = AutoModelForSequenceClassification.from_pretrained(BASE, num_labels=2)
    no_decay = lambda name: name.endswith("bias") or "LayerNorm" in name or "layer_norm" in name
    groups = [{"params": [p for nm, p in model.named_parameters() if not no_decay(nm)],
        "weight_decay": WEIGHT_DECAY},
              {"params": [p for nm, p in model.named_parameters() if no_decay(nm)],
                 "weight_decay": 0.0}]
    opt = torch.optim.AdamW(groups, lr=LR)
    steps_per_epoch = int(np.ceil(n / BATCH)); total = EPOCHS * steps_per_epoch
    check_at = set(np.linspace(0, total, CHECKS + 1).astype(int)[1:])
    rng = np.random.RandomState(seed)
    curve = [(0, (predict(tok, model, Xd) == yd).mean())]
    best = (curve[0][1], 0, copy.deepcopy(model.state_dict()))
    t0, step = time.perf_counter(), 0
    for epoch in range(EPOCHS):
        order = rng.permutation(n)
        for i in range(0, n, BATCH):
            model.train()
            idx = order[i:i + BATCH]
            enc = tok([Xtr[j] for j in idx], padding=True, truncation=True, max_length=MAX_LEN,
                return_tensors="pt")
            loss = model(**enc, labels=torch.tensor(ytr[idx])).loss
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            for g in opt.param_groups:
                g["lr"] = lr_at(step, total)
            opt.step(); opt.zero_grad(); step += 1
            if step in check_at:
                acc = (predict(tok, model, Xd) == yd).mean(); curve.append((step, acc))
                if acc > best[0]:
                    best = (acc, step, copy.deepcopy(model.state_dict()))
    minutes = (time.perf_counter() - t0) / 60
    model.load_state_dict(best[2])
    pt = predict(tok, model, Xt)
    return dict(n=n, seed=seed, curve=np.array(curve), best_step=best[1], dev=best[0], test=(pt ==
        yt).mean(),
                correct=(pt == yt).astype(float), steps=total, minutes=minutes)
```

- `from_pretrained(BASE, num_labels=2)` — the pretrained body plus a NEW randomly initialised two-class head.
- `model.named_parameters()` — every weight with its name, used to exempt biases and LayerNorm from weight decay.
- `loss = model(**enc, labels=...).loss` — given labels, the model computes the cross-entropy itself.
- `loss.backward(); opt.step(); opt.zero_grad()` — PyTorch's three steps: gradients, update, clear (gradients add up otherwise).
- `copy.deepcopy(model.state_dict())` — a full copy of every weight tensor: the best checkpoint, safe from later updates.

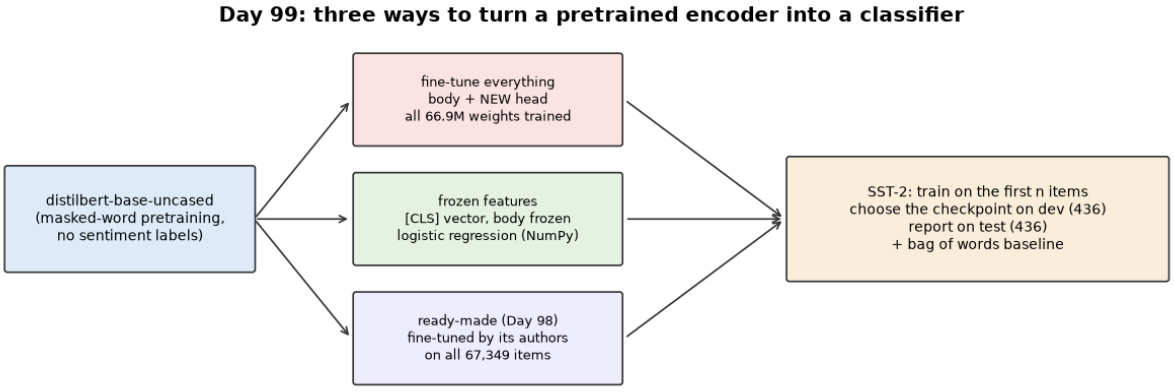
### run_parallel: one process per run

**CODE**

```
def run_parallel(jobs):
    with mp.get_context("spawn").Pool(len(jobs)) as pool:
        return pool.map(fine_tune, jobs)
```

- `mp.get_context("spawn").Pool(len(jobs))` — fresh processes that re-import this script; main() is protected by the __name__ guard.
- `pool.map(fine_tune, jobs)` — results come back in the order of the jobs.

**SYSTEM DIAGRAM**




*Three ways to use a pretrained encoder, all scored on the same test half.*

## Segment 3 — Comparisons

### paired_bootstrap: the difference on the same sentences

**MATH**

$$d^{(r)} = mean(a_{idx^{(r)}}) - mean(b_{idx^{(r)}}), \quad interval = 2.5th \text{ and } 97.5th \text{ percentiles of } d$$

**CODE**

```
def paired_bootstrap(a, b, n_boot=2000, seed=0):
    rng = np.random.RandomState(seed)
    idx = rng.randint(0, len(a), (n_boot, len(a)))
    d = a[idx].mean(axis=1) - b[idx].mean(axis=1)
    return d.mean(), np.percentile(d, 2.5), np.percentile(d, 97.5), (d > 0).mean()
```

- `idx = rng.randint(0, len(a), (n_boot, len(a)))` — one matrix of resampled sentence indices, used for BOTH models.
- `a[idx].mean(axis=1) - b[idx].mean(axis=1)` — the accuracy difference in every resample at once.
- `(d > 0).mean()` — the share of resamples in which the first model is ahead.

### cls_features and logistic: the frozen-encoder baseline

**MATH**

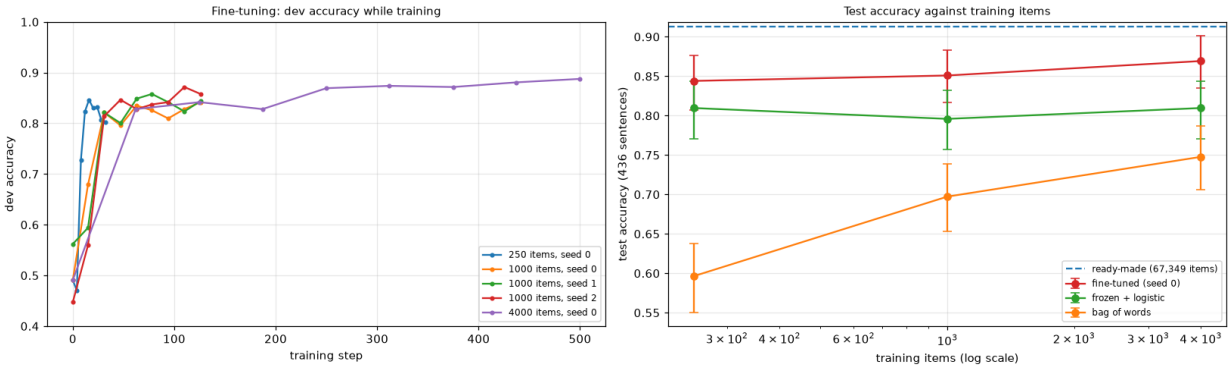
$$p = \sigma(Z\,w + b), \quad \nabla_w = Z^T(p - y)/n + \lambda w$$

**CODE**

```
def cls_features(texts, bs=32):
    tok = AutoTokenizer.from_pretrained(BASE); enc_model = AutoModel.from_pretrained(BASE).eval()
    out = []
    with torch.no_grad():
        for i in range(0, len(texts), bs):
            enc = tok(texts[i:i + bs], padding=True, truncation=True, max_length=128,
                return_tensors="pt")
            out.append(enc_model(**enc).last_hidden_state[:, 0].numpy())
    return np.concatenate(out)
def logistic(F, y, Fd, yd, l2s=(1e-4, 1e-3, 1e-2, 1e-1, 1.0, 10.0, 100.0), steps=500, lr=0.01):
    mu, sd = F.mean(0), F.std(0) + 1e-6
    Z, Zd = (F - mu) / sd, (Fd - mu) / sd
    best, devs = None, {}
    for l2 in l2s:
        w, b = np.zeros(Z.shape[1]), 0.0
        m, v = np.zeros(Z.shape[1] + 1), np.zeros(Z.shape[1] + 1)
        for t in range(1, steps + 1):
            p = 1 / (1 + np.exp(-(Z @ w + b)))
            g = np.r_[Z.T @ (p - y) / len(y) + l2 * w, np.mean(p - y)]
            m = 0.9 * m + 0.1 * g; v = 0.999 * v + 0.001 * g ** 2
            s = lr * (m / (1 - 0.9 ** t)) / (np.sqrt(v / (1 - 0.999 ** t)) + 1e-8)
            w -= s[:-1]; b -= s[-1]
        acc = np.mean(((Zd @ w + b) > 0) == yd); devs[l2] = acc
        if best is None or acc > best[0]:
            best = (acc, l2, w.copy(), b)
    return best, (mu, sd), devs
```

- `AutoModel.from_pretrained(BASE)` — the body only, without any classification head.
- `last_hidden_state[:, 0]` — slicing a (batch, tokens, 768) tensor: position 0, the [CLS] vector, for every text.
- `(F - mu) / sd` — standardise with the training features' mean and spread (also applied to dev and test).
- `Z.T @ (p - y) / len(y) + l2 * w` — the logistic-regression gradient, as one matrix product.

**REAL GRAPH**




*Real graph from the script: dev curves of the five runs, and test accuracy against training items for each method.*

## Quick Syntax Glossary — new or reinforced constructs

- `torch.manual_seed(s)` — fixes PyTorch's random numbers (head initialisation, dropout).
- `model.train() / model.eval()` — dropout on while training, off while measuring.
- `torch.nn.utils.clip_grad_norm_(params, 1.0)` — rescales all gradients together if their norm exceeds 1.
- `opt.param_groups` — the optimizer's groups of weights, each with its own learning rate and decay.
- `copy.deepcopy(obj)` — a copy that shares nothing with the original.
- `loss.backward()` — fills every weight's .grad with the gradient of the loss (PyTorch's version of the backward passes written by hand in earlier weeks).

*End of Day 99 syntax reference. For the experiments and what they showed, see the Day 99 complete guide.*